

When a Line Is Not Enough

Unit 5: Fixing a Model, and Letting a Tree Show You Where to Look

Stat 220 | Statistics for Data Science

Where we left off

- Units 3 and 4 fitted straight lines, read the coefficients, and chose which predictors to keep.
- All of that assumed the straight line was roughly the right shape, and that the spread around it was roughly constant.
- This unit is what you do when it is not: how to tell, what to change, and what to reach for when a line will not do the job at all.

What this unit gives you

Things to know

- what each broken assumption looks like in a residual plot
- when a log fixes spread that grows, and how to read the result
- how to put curves and interactions into a linear model
- how a decision tree is built, one split at a time
- what a tree is good at, and where it falls down
- how to use a tree to find terms worth adding to a regression

Mistakes to stop making

- reading coefficients off a model whose residuals are patterned
- taking logs and then reporting the coefficient in the original units
- adding squared terms everywhere instead of looking first
- reading a tree's splits as effects
- expecting a tree to predict outside the range of the data
- choosing between a tree and a regression on their fit to the same rows

The data for this unit

`lawn_jobs.csv`: 700 completed jobs at a lawn care company.

Column	What it is
<code>minutes</code>	time on the property, the outcome
<code>lot_sqft</code>	size of the lot
<code>slope</code>	flat, moderate, or steep
<code>obstacles</code>	trees, beds and play sets to mow around
<code>gated</code>	1 if the property is behind a gate
<code>crew_size</code>	people on the job, 1 to 3
<code>invoice_id</code> , <code>customer_rating</code>	also recorded

The company schedules by estimated time, so a prediction that is off by a third ruins the afternoon.

What the residuals are telling you

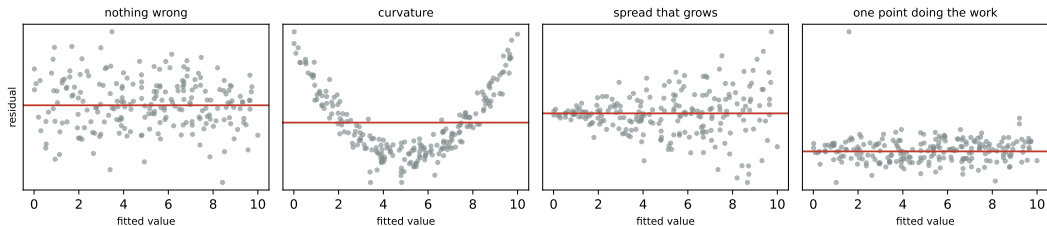
A residual plot is the model showing you what it could not explain. Four things turn up.

What you see	What it means
a shapeless cloud	the shape you fitted is about right
a curve or a U	the relationship bends and the line does not
a fan or a wedge	the spread grows with the size of the prediction
one point far away	a single case is pulling the fit toward itself

Principle: look before you read

The coefficient table does not know whether the shape is right. The residual plot is the only place that shows you.

The four pictures



Only the first one is a model you can read coefficients from as they stand. The other three each have a standard response, and the rest of this unit is those responses.

What to do about each one

Problem	Usual response
the relationship bends spread grows with the fit	add a squared term, or transform the predictor model $\log(y)$ instead of y
one case dominates cases are not independent	find out what it is; report the fit with and without it a different family of models, named on the next slide
the outcome is not a number	also a different family, also next slide

Trap: the quiet failure

Nothing in the output tells you any of this happened. A model with patterned residuals still prints coefficients, p-values and an R^2 to three decimals.

Three things this course points at but does not teach

- **Time series.** Measurements on the same thing over time are not independent: today looks like yesterday. Unit 1 showed what that does to a p-value. Fixing it properly needs models built for it.
- **Spatial statistics.** The same problem in two dimensions: nearby places resemble each other. Mowing times on one street are not independent draws.
- **Generalized linear models.** When the outcome is a yes/no or a count, the model gets a different link and a different spread. Logistic regression, which Unit 10 covers, is one of these.

In practice: what to do when you meet one

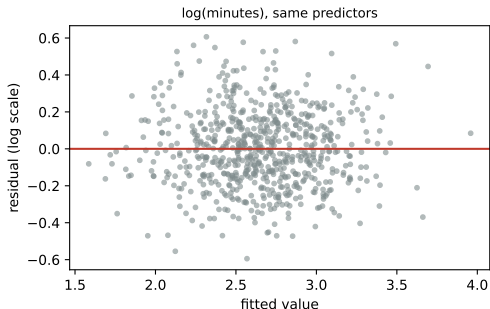
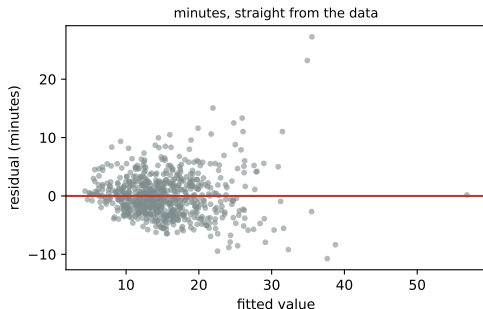
Recognize it, say so, and go find the right tool or the right person. Knowing that your data is a time series is most of the battle; the rest is a course of its own.

Why spread so often grows with the prediction

- A big lawn does not take a fixed number of extra minutes. It takes a percentage longer, and the things that go wrong on it cost a percentage too.
- When effects multiply rather than add, the spread around a big prediction is bigger than the spread around a small one. That is the fan.
- Taking logs turns multiplication into addition, which is exactly the shape a linear model is built for.

$$y = a \cdot b^x \cdot \varepsilon \quad \implies \quad \log y = \log a + x \log b + \log \varepsilon$$

The same jobs, before and after



Left: minutes on the raw scale. The spread at a 40-minute prediction is nearly twice the spread at a 10-minute one. Right: the same predictors against $\log(\text{minutes})$. The fan is gone and R^2 rises from 0.69 to 0.74.

Reading a coefficient after a log

- On a logged outcome, a coefficient b is a *percentage*: roughly $100(e^b - 1)\%$ for each one-unit change in the predictor.
- Gated properties have a coefficient of 0.26, which is about **30% longer**, not 0.26 minutes longer.
- If you log the predictor too, the coefficient is a percentage for a percentage. Here the coefficient on $\log(\text{lot size})$ is 0.52: a lot 10% bigger takes about 5% longer.

Trap: reporting in the wrong units

“Gating adds 0.26” is not a sentence about minutes, and a manager will read it as one. Convert before you report.

Two things to know before you take a log

- **Zeros and negatives have no log.** Counts that include zero need a different answer, not $\log(y + 1)$ chosen because it ran without an error.
- **Exponentiating a fitted value gives you back a median, not a mean.** For most operational questions that is fine, and it is worth knowing which one you handed over.
- The model is now about $\log(\text{minutes})$. Every interval and every prediction comes back on that scale until you convert it.

Your turn #1

Your turn: which fix fits which picture?

Match each residual plot to its response:

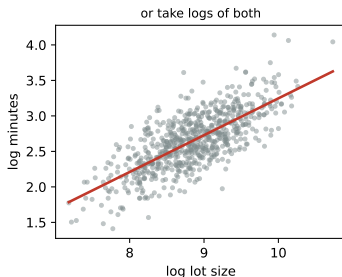
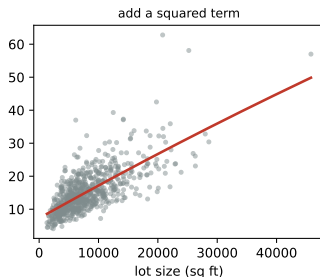
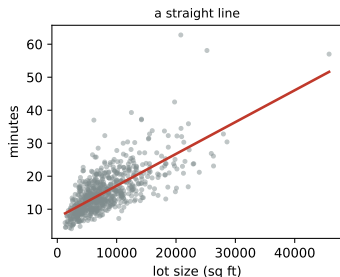
- 1 The residuals fan out as the fitted value grows.
- 2 The residuals form a clear arc, negative at both ends.
- 3 One residual sits five times further from zero than any other.

Decide with a neighbour, and say what you would check before doing anything to number 3.

Your turn #1: answer

- **(1) Fan.** Model the log of the outcome, then report in percentages.
- **(2) Arc.** The shape is wrong, not the spread. Add a squared term or transform the predictor.
- **(3) One point.** Find out what it is first. A data entry error gets fixed, a real unusual job gets reported both ways, and neither gets deleted because it was inconvenient.

Bending a line on purpose



The same 700 jobs three ways. A straight line misses the shape at both ends. A squared term follows it. Taking logs of both sides straightens the relationship instead of bending the model, which is usually the better move when the cause of the curve is multiplicative.

The two tools you already have

- **Polynomial terms.** Add x^2 , and the fitted curve can bend once. Add x^3 and it can bend twice. Each is an ordinary column, and the model is still linear regression.
- **Interactions.** Add x_1x_2 , and the effect of one predictor is allowed to depend on the other. Steep lots may cost more time only when they are also large.
- Both are choices *you* make about shape, before fitting.

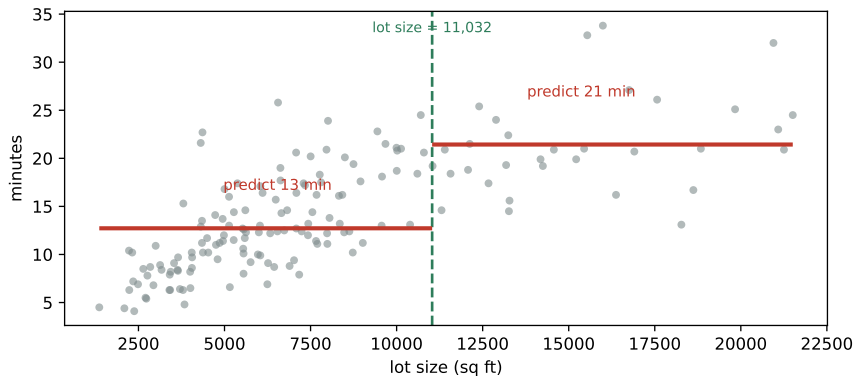
Principle: the catch

A squared term only helps if you suspected a curve in that variable. An interaction only helps if you suspected those two variables interact. With a dozen predictors there are far more possibilities than you can check by hand.

A different idea entirely

- Stop writing an equation. Instead, ask a sequence of yes/no questions and read the answer off the end.
- *Is the lot bigger than 11,000 square feet? If yes, is it behind a gate? If yes, predict 38 minutes.*
- That is a **decision tree**. The questions are chosen by the data, not by you, which is the point.

One split



Cut the jobs into two groups at a lot size of 11,000 square feet. Predict the average of each group. The prediction is now a step function with two steps.

How the cut is chosen

For a candidate cut, score it by how much of the spread is left inside the two groups:

$$\text{score} = \sum_{\text{left}} (y_i - \bar{y}_{\text{left}})^2 + \sum_{\text{right}} (y_i - \bar{y}_{\text{right}})^2$$

- Try every cut point of every predictor. Keep the one with the smallest score.
- Then do the same thing again inside each group, and again inside those.
- Nothing here is clever. It is a search, and the machine does it exhaustively.

Principle: greedy, one step at a time

Each split is chosen as if it were the last one. The tree never goes back and reconsiders an earlier cut, which is why a slightly different sample can produce a rather different tree.

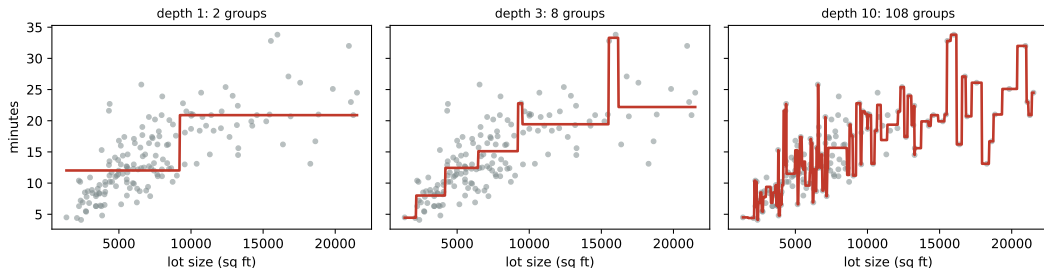
The same arithmetic, in numbers

Three candidate cuts on lot size, scored on the same 160 jobs:

Cut at	left average	right average	spread left over
5,000 sq ft	9 min	21 min	7,900
11,000 sq ft	12 min	28 min	6,500
20,000 sq ft	15 min	37 min	7,200

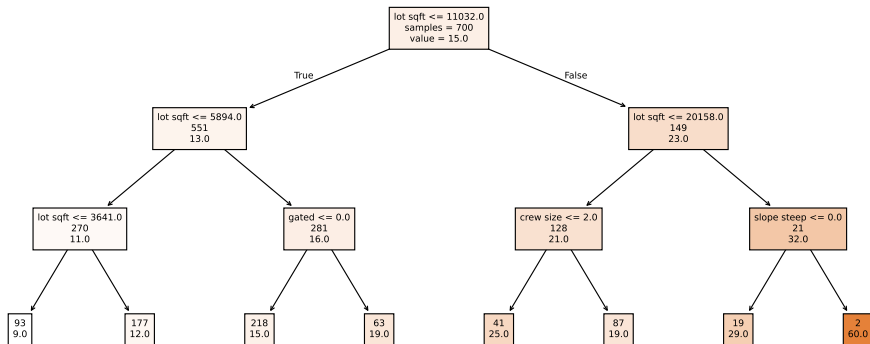
The middle cut wins, so that is the first question the tree asks. The machine does this for every cut point of every variable before choosing.

Keep splitting



One split gives two steps. Three levels give eight groups and a recognisable staircase. Ten levels give a step for almost every handful of jobs, which is the tree memorising this sample.

Reading a fitted tree



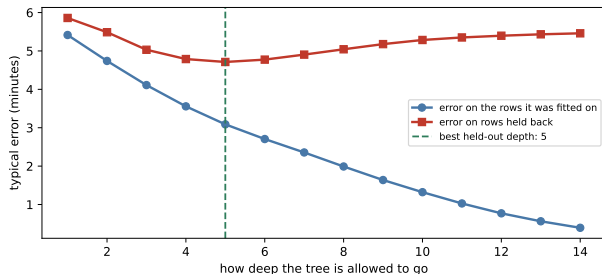
The same tree, out loud

- The first question is lot size, at about 11,000 square feet. Size matters more than anything else here, which matches the regression.
- Inside the small-lot branch, the next question is **the gate**. Gated properties take noticeably longer at the same size.
- Inside the large-lot branch, the tree asks about **crew size** and then about **steep slope**. Slope appears only on large lots.
- Each box at the bottom holds a group of jobs and predicts their average.

Principle: what a tree is

A set of rules that partition the data, and one number per region. No coefficients, no equation, and nothing that can be read as the effect of a variable.

How deep to go



- Error on the fitted rows falls with every level and keeps falling. A deep enough tree fits them exactly.
- Held-out error bottoms out around depth 5 and then climbs.
- Depth is this model's complexity dial, and it is chosen the same way as everything in Unit 4: on data the tree did not see.

What trees are good at, and bad at

Good at	Bad at
thresholds, where something changes at a value	smooth trends, which it approximates with stairs
interactions, free of charge, because every split happens inside an earlier split	giving you an effect size with an interval
predictors on any scale, with no transformation needed	stability: a few different rows can change the top split
mixed types of predictor in one model	anything outside the range of the data

The extrapolation problem, in one line

- A tree predicts the average of the training jobs in a region. For a lot bigger than any it has seen, it returns the average of the biggest group it knows.
- A line at least continues in a direction. A tree flattens out completely.
- Neither is trustworthy out there, but they fail differently, and the tree fails silently.

Forests and boosting, in one slide

- A single tree is unstable. **Random forests** fit many trees to resampled data and average them. **Boosting** fits trees in sequence, each correcting the last.
- Both usually predict better than one tree, and both give up the one thing a single tree had: you could read it.
- We are not going further than naming them. Unit 2 put them on the menu, and this is where they come from.

Your turn #2

Your turn: what would the tree do?

A tree is fitted to predict mowing time. Its first split is *lot size above 11,000 sq ft*, and inside the small-lot branch it splits on *gated*.

A new job comes in: a 2,000 square foot lot, behind a gate. What does the tree predict, roughly, and what would a regression with a gate coefficient predict differently?

Your turn #2: answer

- The tree sends the job left at the first question, then right at the gate question, and predicts the average of the small gated jobs it has seen.
- A regression adds a gate coefficient to whatever the size term gives, so the gate costs the same percentage on a tiny lot as on a large one.
- Which is right depends on the gate: if it is a fixed walk from the street, the tree's version is closer. That is a question about mowing, not about statistics.

Using a tree to improve a regression

- A tree will not give you an effect with an interval, so it cannot be the thing you report when the question is how much something matters.
- But it searches shapes you would never check by hand, and its splits are hypotheses written in a readable form.
- So: fit the tree, read the structure, and put what it found into the regression.

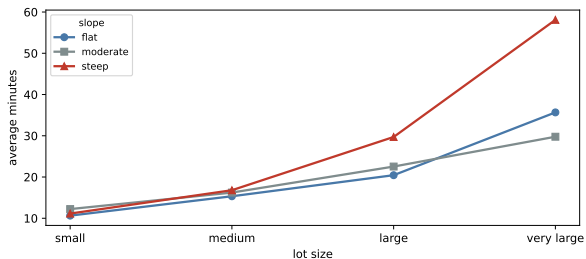
Principle: scout and report

Let the tree tell you where to look. Let the regression tell you how much, and how sure.

What this tree was telling us

- **A split on gated, inside one size branch.** The gate costs time, and it is worth a term of its own.
- **A split on steep slope only inside the large-lot branch.** Slope seems to matter on big lots and not on small ones. In regression language, that is an interaction between size and slope.
- **Splits on lot size at several places.** The relationship with size is not a straight line, which the log already told us.

The hint, checked



Average minutes by lot size band and slope.

- On small and medium lots the three slopes cost about the same.
- On large and very large lots, steep pulls away from the others.
- That is what the tree saw when it split on slope only inside the large branch.

Put it in the model

Add one interaction, between $\log(\text{lot size})$ and steep, to the log model:

Model	R^2	AIC	cross-validated error
line on raw size			0.236
log size, log minutes	0.742	-215	0.208
plus size $\times$ steep	0.745	-223	0.207
the tree on its own (depth 5)			0.253

- The interaction the tree pointed at is real: $p = 0.0016$, and AIC improves by 8.
- The regression that took the hint predicts better than the tree that gave it.

Why end here

- The tree is a fast, honest search for structure you did not think to look for.
- The regression turns what it finds into a number with an interval, in units somebody can act on.
- Used together they cover each other's weakness, and you can defend every term in the final model by saying where it came from.

Trap: the step you cannot skip

A term suggested by a tree was still suggested by looking at the data. Check it on held-out rows, and say in the write-up that this is where it came from.

LLM check: mistake or not? (1 of 2)

LLM check: we asked about a fan in the residuals

We told it: “My residuals fan out as the fitted values grow. What should I do?”

It answered: “Remove the outliers at the high end and refit. That will stabilise the variance and improve your R^2 .”

Mistake, or no mistake? Decide before we click.

LLM check: mistake or not? (1 of 2)

LLM check: we asked about a fan in the residuals

We told it: “My residuals fan out as the fitted values grow. What should I do?”

It answered: “Remove the outliers at the high end and refit. That will stabilise the variance and improve your R^2 .”

Mistake, or no mistake? Decide before we click.

Trap: verdict: mistake

A fan is not an outlier problem. The large cases are not errors, they are simply more variable, and deleting them changes the question the model answers. Model the log instead.

LLM check: mistake or not? (2 of 2)

LLM check: we asked it to read a tree

We told it: “My tree splits on gated right after lot size. What does that tell me?”

It answered: “Gating is the second most important driver of mowing time. The split suggests it matters within a size range, so it may be worth testing a gate term, or a gate by size interaction, in a regression where you can put a number on it.”

Mistake, or no mistake?

LLM check: mistake or not? (2 of 2)

LLM check: we asked it to read a tree

We told it: “My tree splits on gated right after lot size. What does that tell me?”

It answered: “Gating is the second most important driver of mowing time. The split suggests it matters within a size range, so it may be worth testing a gate term, or a gate by size interaction, in a regression where you can put a number on it.”

Mistake, or no mistake?

Principle: verdict: no mistake

The second sentence is exactly the move this unit ends on. The phrase “second most important” is loose, but it stops short of calling the split an effect.

What to do when a line is not enough

- 1 **Plot the residuals** against the fitted values, every time.
- 2 **Name what you see:** cloud, curve, fan, or one point.
- 3 **Curve:** add a squared term, or transform the predictor.
- 4 **Fan:** model the log of the outcome, and report in percentages.
- 5 **One point:** find out what it is before touching it.
- 6 **Not independent, or not a number:** name the right family and get help.
- 7 **Out of ideas:** fit a shallow tree and read its splits for terms to try.
- 8 **Check any term you found this way** on data the search did not touch.

The traps, all in one place

Trap: the ones that bite most often

- Reading coefficients from a model whose residuals are patterned.
- Reporting a coefficient from a logged model in the original units.
- Using $\log(y + 1)$ on zeros because it ran without complaining.
- Deleting an inconvenient point and not saying so.
- Calling a tree's split an effect, or its variable ranking an effect size.
- Asking a tree to predict outside the range of its training data.
- Comparing a tree and a regression on the rows both were fitted on.
- Adding a tree-suggested term and reporting its p-value as if you had planned it.

What you should be able to do with software

You should be able to	What you would reach for
Plot residuals against fitted values	<code>plt.scatter(fit.fittedvalues, fit.resid)</code>
Model a logged outcome	<code>smf.ols("np.log(y) ~ x", data)</code>
Add a curve	<code>smf.ols("y ~ x + I(x**2)", data)</code>
Add an interaction	<code>smf.ols("y ~ x * group", data)</code>
Fit a shallow tree	<code>DecisionTreeRegressor(max_depth=3).fit(X, y)</code>
Read the tree	<code>export_text(tree, feature_names=...)</code>
Choose the depth	<code>cross_val_score</code> over a few depths

Where we go next

- That closes the modelling half of the course. You can fit a model, read it, check it, repair it, choose between candidates, and predict with honest error bars.
- **Unit 6** starts the machinery underneath all of it: probability, and what chance alone produces.
- Everything from here gives a reason for the tools you have been using, and then puts them to work on outcomes that are not continuous numbers.